

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



BÀI TẬP LỚN MÔN XÁC SUẤT THỐNG KÊ

Đề tài Xử lý dữ liệu

Phân Tích và Dự Đoán Giá Phát Hành GPU

Dựa Trên Các Thông Số Kỹ Thuật

GVHD: Huỳnh Thái Duy Phương

Lớp: L03 — Nhóm: 16

SV: Trần Gia Tường - 2313835

Trần Nhật - 2412490

Trương Minh Triết - 2413604

Vũ Đại Dương - 2410638

TP. HỒ CHÍ MINH, 2026

DANH SÁCH THÀNH VIÊN

STT	Họ và tên	MSSV	Đóng góp
1	Trần Gia Tường	2313835	100%
2	Trần Nhật	2412490	100%
3	Trương Minh Triết	2413604	100%
4	Vũ Đại Dương	2410638	100%



Mục lục

1	Tổng quan đề tài	1
2	Cơ sở Lý thuyết	1
2.1	Định giá GPU: Hiệu năng, thương hiệu và thời gian	1
2.2	Bản chất dữ liệu	2
2.3	Tiền xử lý dữ liệu	3
2.3.1	Lọc và làm sạch dữ liệu thô	3
2.3.2	Loại bỏ tất cả dữ liệu thiếu giá	4
2.4	Thu hẹp không gian đặc trưng	4
2.5	Loại bỏ giá trị ngoại lai (Outliers)	7
2.6	Chia tập mẫu thành tập huấn luyện và tập kiểm tra	7
2.7	Chiến lược điền khuyết theo vai trò biến	7
2.8	Chuẩn hóa và chuyển đổi biến	8
2.9	Xây dựng mô hình thống kê	9
2.9.1	Xây dựng và lựa chọn mô hình Hồi quy Tuyến tính Bội	9
2.9.2	Các kiểm định thống kê và chẩn đoán mô hình	10
2.9.3	Đánh giá mô hình và phân tích tầm quan trọng của biến	10
2.10	Sơ đồ tổng quan quy trình phân tích	11
3	Thực thi Code và Xử lý dữ liệu	12
3.1	Bước 1: Tải dữ liệu và Chọn lọc 8 biến	12
3.2	Bước 2: Làm sạch, xóa khuyết giá và ép kiểu	12
3.3	Bước 3: Xử lý ngoại lai (IQR)	13
3.4	Bước 4: Chia tập dữ liệu Train/Test (90:10)	13
3.5	Bước 5: Điền khuyết trên Train (Regex, Mode, Median)	14
3.6	Bước 6: Biến đổi Logarit Memory_Bandwidth	15
3.7	Bước 7: Xây dựng & so sánh MLR cơ bản vs Log-Linear	15
3.8	Bước 8: Kiểm định thống kê (ANOVA, BP, Shapiro-Wilk)	16
3.9	Bước 9: Đánh giá mô hình trên tập Test ngoài mẫu	16
3.10	Bước 10: Chuẩn hóa Z-Score & Phân tích Feature Importance	17
3.11	Bước 11: Dự báo giá GPU mới (Validation in-the-wild)	18



3.12	Bước 12: Trục quan hóa kết quả	18
4	Kết quả Thu được	19
4.1	Phân phối biến phụ thuộc	19
4.2	Chênh lệch giá NVIDIA và AMD	19
4.3	Yếu tố nào quyết định giá GPU?	20
4.3.1	Bảng hệ số hồi quy Log-Linear	20
4.3.2	Tầm quan trọng tương đối của các biến	21
4.3.3	Khả năng dự đoán của mô hình	22
4.3.4	Kiểm định giả định và chẩn đoán mô hình	23
5	Kết luận	24
5.1	Đánh giá kết quả nghiên cứu	24
5.2	Hạn chế và đề xuất cải tiến	25
5.3	Ý nghĩa thực tiễn	25

1 Tổng quan đề tài

Đề tài sử dụng bộ dữ liệu **All_GPUs.csv** từ tập “*Computer Parts (CPUs and GPUs)*” của tác giả Ilias Kkaf trên Kaggle, chứa thông số kỹ thuật chi tiết của hơn 3.400 card đồ họa (GPU) từ các nhà sản xuất NVIDIA, AMD và Intel qua nhiều thế hệ sản phẩm. Bộ dữ liệu thô gồm 34 thuộc tính.

Báo cáo hướng đến hai mục tiêu chính:

- **Thứ nhất**, xây dựng mô hình hồi quy tuyến tính bội (Multiple Linear Regression) để định lượng tác động của từng thông số kỹ thuật lên giá phát hành GPU, với phương trình tổng quát

$$Y = \beta_0 + \beta_1 X_1 + \dots + \beta_k X_k + \varepsilon$$

Trong đó Y là giá phát hành, các X_i là thông số kỹ thuật và ε là sai số ngẫu nhiên. Mô hình sẽ được tinh chỉnh (bao gồm biến đổi logarit nếu cần) nhằm thỏa mãn các giả định thống kê và tối ưu khả năng giải thích (R^2).

- **Thứ hai**, áp dụng phân tích phương sai một chiều (ANOVA) để kiểm định xem có sự khác biệt có ý nghĩa thống kê về giá trung bình giữa GPU của NVIDIA và AMD hay không.

Sau khi hoàn thành, nhóm kỳ vọng đạt được: (i) mô hình hồi quy dạng $\log(\hat{Y}) = \beta_0 + \beta_1 X_1 + \dots + \beta_k X_k$ có khả năng dự đoán giá GPU với $R^2 \geq 70\%$, đồng thời cho phép so sánh tầm quan trọng tương đối của từng thông số; (ii) xác định được các yếu tố định giá chủ đạo, kỳ vọng TDP, dung lượng bộ nhớ và hãng sản xuất có tác động mạnh nhất; (iii) bằng chứng thống kê về mức chênh lệch giá giữa NVIDIA và AMD thông qua kiểm định ANOVA; và (iv) cơ sở thực tiễn giúp người mua đánh giá mức giá GPU có hợp lý so với thông số kỹ thuật của nó hay không.

2 Cơ sở Lý thuyết

2.1 Định giá GPU: Hiệu năng, thương hiệu và thời gian

Card đồ họa (GPU) là một trong số ít linh kiện điện tử mà giá bán và thông số kỹ thuật thường không tỷ lệ tuyến tính với nhau. Một GPU flagship của NVIDIA có thể đắt gấp đôi một card AMD cùng thế hệ dù chênh lệch hiệu năng không tương

xúng — phần còn lại là *định giá thương hiệu, chiến lược phân khúc và hệ sinh thái phần mềm*. Điều đó đặt ra câu hỏi không đơn giản: **giá phát hành GPU thực sự được cấu thành bởi những yếu tố nào?**

Đề tài tiếp cận câu hỏi này qua hai góc nhìn bổ trợ:

- **Hỏi quy đa biến:** Dự đoán giá và định lượng tác động đồng thời của TDP, xung nhịp, dung lượng bộ nhớ, năm phát hành và thương hiệu lên giá. Mô hình cho phép trả lời: nếu tăng VRAM thêm 1 độ lệch chuẩn trong khi giữ nguyên mọi thông số khác, giá GPU thay đổi bao nhiêu phần trăm?
- **Kiểm định ANOVA:** Kiểm tra riêng biệt xem khoảng cách giá NVIDIA–AMD có ý nghĩa thống kê, trước khi đưa biến thương hiệu vào mô hình đa biến như một biến giả.

Thách thức nằm ở chỗ phần lớn GPU trong bộ dữ liệu không có giá niêm yết chính thức ($\approx 84\%$ thiếu giá), và 34 thuộc tính ban đầu chứa nhiều thông tin dư thừa. Các phần tiếp theo mô tả phương pháp luận để giải quyết những thách thức đó.

2.2 Bản chất dữ liệu

Dữ liệu phần cứng GPU mang một số đặc thù quan trọng cần lưu ý trước khi tiến hành xử lý và xây dựng mô hình:

- **Đặc tính rời rạc của bộ nhớ:** Các biến số như băng thông bộ nhớ (`Memory_Bus`) và dung lượng bộ nhớ (`Memory`) mang bản chất là các biến **rời rạc**. Chúng chỉ tồn tại theo các chuẩn phần cứng nhất định (ví dụ: bus 64/128/256/512 bit; dung lượng 2/4/8/16 GB).
- **Sự phân mảnh nhãn thương hiệu (ATI và AMD):** ATI Technologies là tên cũ của bộ phận GPU của AMD trước khi bị thôn tính vào năm 2006. Hiện tại, bộ dữ liệu thô tồn tại song song hai nhãn "ATI" và "AMD" cho cùng một dòng sản phẩm. Hai nhãn này thực chất cần được gộp chung thành một thực thể thương hiệu duy nhất.
- **Sự khuyết thiếu dữ liệu giá của Intel:** Toàn bộ các GPU của hãng Intel trong bộ dữ liệu thô đều thiếu thông tin về biến phụ thuộc là giá phát hành (`Release_Price`), do đó các sản phẩm này không thể được sử dụng trong việc xây dựng mô hình định giá và cần loại bỏ hoàn toàn.

- **Các sản phẩm có cấu trúc định giá dị biệt:** Dữ liệu chứa một số nhóm sản phẩm có cơ chế định giá nằm ngoài quy luật phần cứng thông thường và không phụ thuộc vào các thuộc tính có trong bộ dữ liệu. Điển hình như: GPU Workstation (Quadro/FirePro) định giá theo chứng chỉ phần mềm; GPU Titans bị thổi giá do sản xuất limited; các cấu hình kép (Crossfire/Dual Core) thực chất là nhiều GPU bị cộng dồn thông số nhưng bán chung với nhau; các bản độ tản nhiệt nước cao cấp (Hydro Copper); hoặc sản phẩm tin đồn chưa từng ra mắt (not released). Nếu không được thanh lọc cẩn thận, nhóm dữ liệu dị biệt này sẽ gây nhiễu nặng nề cho phương trình dự báo của thị trường Consumer.

2.3 Tiền xử lý dữ liệu

2.3.1 Lọc và làm sạch dữ liệu thô

Trước khi xử lý ngoại lai hay điền khuyết, bộ dữ liệu thô cần trải qua một chuỗi thao tác làm sạch để đảm bảo tính nhất quán và phù hợp với mục tiêu phân tích:

1. Thay thế các chuỗi rỗng, dấu gạch ngang và "N/A" thành giá trị NA thực sự trong R để dễ dàng nhận diện và xử lý sau này.
2. Hợp nhất toàn bộ giá trị "ATI" trong cột `Manufacturer` thành "AMD".
3. Loại bỏ các dòng có `Manufacturer == "Intel"`.
4. Tách Năm từ cột `Release_Date`, đổi tên thành `Release_Year` và xóa cột `Release_Date` gốc. Việc này không chỉ chuẩn hóa định dạng ngày tháng mà còn loại bỏ nhiễu từ ngày và tháng không liên quan đến định giá (chu kỳ giá GPU tính theo năm).
5. **Trích xuất số và ép kiểu dữ liệu.** Các trường số trong dữ liệu gốc thường đi kèm đơn vị văn bản (như "141 Watts", "8 GB"). Nhóm sử dụng biểu thức chính quy (Regular Expression) để trích xuất chính xác phần số, sau đó thực hiện ép kiểu `as.numeric()`. Quy trình này đảm bảo các giá trị không hợp lệ hoặc chuỗi rỗng được chuyển thành NA thực sự, tạo tiền đề cho các bước xử lý ngoại lai và điền khuyết.
6. Chuyển đổi các biến định danh (`Manufacturer`, `Memory_Type`) sang kiểu `Factor`. Điều này không chỉ giúp R tự động sinh biến giả (dummy variables) trong mô hình hồi quy mà còn đảm bảo rằng các biến này được xử lý đúng

cách như các nhóm phân loại thay vì số liên tục.

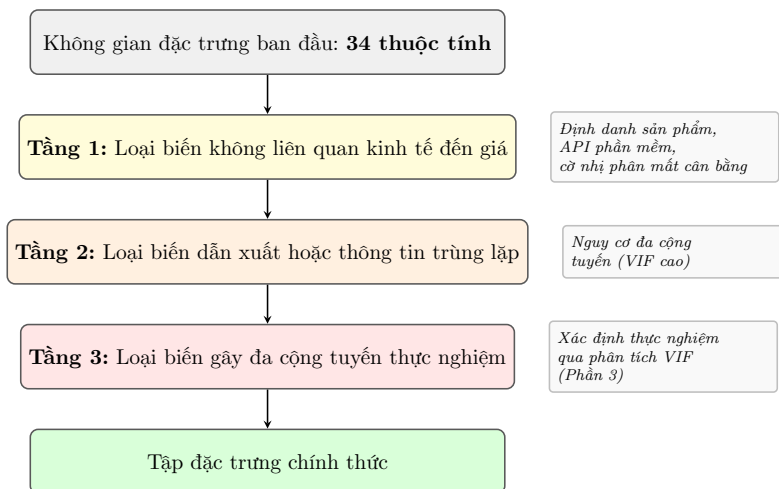
2.3.2 Loại bỏ tất cả dữ liệu thiếu giá

Nguyên tắc rất quan trọng: Trong mô hình dự đoán, Biến mục tiêu Y (Giá phát hành) là “vua”. Bất kỳ dòng nào không có Y đều là “rác” đối với quá trình xây dựng mô hình. Với lại, `Release_Price` có tỷ lệ khuyết rất cao ($\approx 84\%$). Việc điền khuyết sẽ tạo ra vòng lặp logic khi mô hình tự học từ dữ liệu giả do chính nó tạo ra.

Do đó, nhóm quyết định **loại bỏ hoàn toàn** các quan sát thiếu giá, giữ lại những mẫu có giá gốc để xây dựng mô hình hồi quy.

2.4 Thu hẹp không gian đặc trưng

Đưa toàn bộ 34 thuộc tính vào mô hình không những không cải thiện khả năng dự đoán mà còn làm giảm chất lượng: biến trùng lặp gây đa cộng tuyến, biến không liên quan thêm nhiễu và tăng phương sai ước lượng. Nhóm giải quyết vấn đề này bằng quy trình sàng lọc **ba tầng khách quan** — mỗi tầng nhắm đến một nguồn nhiễu hoàn toàn khác nhau:



Hình 1: Chiến lược lọc ba tầng: thu hẹp không gian đặc trưng.

Tầng 1 và Tầng 2 có thể xác định hoàn toàn bằng lý luận trước khi nhìn vào dữ liệu. Tầng 3 cần tính giá trị VIF thực tế và được giải quyết trong Phần 3.

Bảng 2: Tầng 1 — 14 biến bị loại do không có ý nghĩa kinh tế

Biến bị loại	Nội dung	Lý do cụ thể
Name, Architecture	Nhãn định danh sản phẩm	Không phải đặc tính kỹ thuật định lượng, không có cơ chế trực tiếp tác động đến chi phí sản xuất.
DVI_Connection, HDMI_Connection, VGA_Connection	Đặc tả cổng đầu ra hiển thị	Số lượng và loại cổng phụ thuộc vào phân khúc sản phẩm, không phản ánh hiệu năng GPU.
Direct_X, Open_GL	Phiên bản API đồ họa	Hoàn toàn phụ thuộc vào driver phần mềm và thể hệ kiến trúc, không phản ánh sức mạnh phần cứng thực.
Best_Resolution, Resolution_WxH, Power_Connector	Thông số hiển thị và loại đầu nối nguồn	Dẫn xuất gián tiếp từ cấu hình phần cứng, không mang thêm thông tin độc lập về giá trị GPU.
Dedicated, Integrated, Notebook_GPU, SLI_Crossfire	Số nhĩ phân mất cân bằng nặng	Gần như toàn bộ mẫu thuộc cùng một nhĩ (ví dụ: >98% là GPU dedicated), biến không mang thông tin phân biệt.

Bảng 3: Tầng 2 — 10 biến bị loại do thông tin dư thừa (nguy cơ đa cộng tuyến)

Biến bị loại	Quan hệ dư thừa	Biến giữ lại liên quan
Memory_Bus	Chiều rộng bus (bit) chỉ là một trong hai thừa số của băng thông: $\text{Memory_Bandwidth} = \text{Memory_Bus} \times \text{Memory_Speed}$	Memory_Bandwidth
Pixel_Rate	$= \text{ROPs} \times \text{Core_Speed}$ — tốc độ lấp đầy pixel, dẫn xuất trực tiếp.	Core_Speed
Texture_Rate	$= \text{TMUs} \times \text{Core_Speed}$ — tốc độ ánh xạ texture, dẫn xuất trực tiếp.	Core_Speed



Biến bị loại	Quan hệ dư thừa	Biến giữ lại liên quan
Shader, TMUs, ROPs	Số đơn vị tính toán tăng theo xung nhịp và thể hệ kiến trúc; tương quan cấu trúc cao với Core_Speed .	Core_Speed
Memory_Speed	Tốc độ xung nhịp VRAM; cùng với Memory_Bus xác định băng thông.	Memory_Bandwidth
Process	Kích thước tiến trình litho (nm) giảm đơn điệu theo năm phát hành; tác động đã được nắm bắt bởi Release_Date .	Release_Date
L2_Cache	Dung lượng cache tỷ lệ với băng thông bộ nhớ và độ phức tạp lõi xử lý.	Memory_Bandwidth , Core_Speed
Boost_Clock	Xung nhịp tăng tốc tự động; tương quan cực cao với Core_Speed — đưa cả hai vào mô hình gây đa cộng tuyến nghiêm trọng.	Core_Speed

Sau khi lọc qua Tầng 1 và Tầng 2, ta còn lại các biến:

Bảng 4: Danh sách biến sử dụng trong mô hình và vai trò tương ứng

Biến	Kiểu	Mô tả	Vai trò
Manufacturer	Nominal	Hãng sản xuất (NVIDIA, AMD)	Độc lập (phân loại)
Memory_Type	Nominal	Chuẩn VRAM (GDDR5, GDDR3, DDR3, HBM...)	Độc lập (phân loại)
Memory_Bandwidth	Liên tục	Băng thông bộ nhớ (GB/s)	Độc lập (số)
Memory	Rời rạc	Dung lượng VRAM (GB/MB)	Độc lập (số)
Max_Power	Liên tục	Công suất tiêu thụ (Watt)	Độc lập (số)
Core_Speed	Liên tục	Tốc độ xung nhịp lõi (MHz)	Độc lập (số)
Release_Year	Số nguyên	Năm phát hành (parse và chuyển đổi từ Release_Date)	Độc lập (số)
Release_Price	Liên tục	Giá phát hành (USD)	Phụ thuộc Y

2.5 Loại bỏ giá trị ngoại lai (Outliers)

Nhóm sử dụng phương pháp IQR (Tứ phân vị) để phát hiện và loại bỏ ngoại lai dựa trên định nghĩa: $Q_1 - 1,5 \times \text{IQR}$ và $Q_3 + 1,5 \times \text{IQR}$.

Một nguyên tắc dễ bị vi phạm: **ranh giới IQR phải được tính trên tập dữ liệu gốc** trước khi điền khuyết. Nếu điền trung vị cho hàng trăm ô trống trước, khoảng tứ phân vị sẽ co lại giả tạo, dẫn đến nhiều quan sát bình thường bị gán nhầm là ngoại lai và phương pháp Tukey mất đi ý nghĩa.

Do đó, nhóm thực hiện loại bỏ outliers trước khi điền khuyết, đảm bảo rằng ranh giới IQR phản ánh đúng sự phân bố tự nhiên của dữ liệu gốc.

2.6 Chia tập mẫu thành tập huấn luyện và tập kiểm tra

Sau khi loại bỏ ngoại lai, tập dữ liệu được chia ngẫu nhiên theo tỷ lệ **90:10** thành tập huấn luyện (`train_data`) và tập kiểm tra (`test_data`). Trong R, ta sử dụng hàm `set.seed(2026252)` để đặt hạt giống cố định, đảm bảo kết quả nhất quán hoàn toàn sau mỗi lần chạy code nhưng vẫn đảm bảo tính ngẫu nhiên trong việc chọn mẫu.

Việc chia tập được thực hiện *trước* bước điền khuyết. Đây là yêu cầu bắt buộc: nếu điền khuyết toàn bộ dữ liệu rồi mới chia, các tham số thống kê (Median, Mode) sẽ được tính trên cả tập kiểm tra, khiến thông tin của tập kiểm tra rò rỉ vào quá trình xây dựng mô hình — vi phạm nguyên tắc đánh giá ngoài mẫu.

2.7 Chiến lược điền khuyết theo vai trò biến

Việc xử lý các giá trị khuyết, không có phương pháp điền khuyết “một cho tất cả”. Lựa chọn phương pháp phụ thuộc vào vai trò của biến trong mô hình và đặc điểm phân phối của nó:



Bảng 5: Chiến lược điền khuyết dựa trên vai trò và đặc điểm của biến

Biến	Phương pháp	Cơ sở lý luận
Memory (bước 1 — ưu tiên)	Trích xuất từ tên GPU (Regex)	Tên sản phẩm thường ghi rõ dung lượng VRAM (ví dụ: “8 GB”); regex <code>\b(\d+)\s*(GB MB)\b</code> khôi phục giá trị gốc chính xác hơn bất kỳ phép nội suy thống kê nào.
Memory (bước 2 — dự phòng)	Yếu vị (Mode)	Áp dụng khi tên GPU không chứa thông tin VRAM; đảm bảo giá trị điền khuyết luôn là một chuẩn phần cứng có thật (rời rạc).
Core_Speed, Max_Power, Memory_Bandwidth, Release_Year	Trung vị (Median)	Bền vững với các giá trị cực đại (flagship GPU) làm lệch trung bình.
Memory_Type, Manufacturer	Yếu vị (Mode)	Bảo toàn cấu trúc và tần suất của các nhóm định danh.

Lưu ý chống rò rỉ dữ liệu (Data Leakage): Toàn bộ tham số điền khuyết (Median, Mode) được tính toán *chỉ* từ tập huấn luyện (`train_data`), sau đó áp dụng sang tập kiểm tra (`test_data`). Điều này ngăn thông tin của tập kiểm tra ảnh hưởng ngược lên quá trình xây dựng mô hình.

2.8 Chuẩn hóa và chuyển đổi biến

Các biến độc lập trải qua ba bước biến đổi, áp dụng ở các giai đoạn khác nhau trong pipeline (Bảng 6):

Bảng 6: Tóm tắt biến đổi và mã hóa biến theo pipeline thực tế

Biến	Loại	Xử lý
Memory_Bandwidth	Liên tục	Log-transform: <code>log_Memory_Bandwidth = log(Memory_Bandwidth)</code> , dùng trong mô hình log-linear và mô hình chuẩn hóa.



Biến	Loại	Xử lý
Max_Power, Memory, Core_Speed, Release_Year, log_Memory_Bandwidth	Số	Z-score $z = (x - \bar{x})/s$, tham số tính từ <code>train_data</code> . Chỉ áp dụng cho mô hình chuẩn hóa (phân tích tầm quan trọng biến); mô hình dự báo chính dùng giá trị thô.

Log-transform Memory_Bandwidth. Bằng thông bộ nhớ có phân phối lệch phải mạnh do các flagship GPU tạo đuôi dài. Logarithm tự nhiên nén đuôi phân phối, ổn định phương sai phần dư và tuyến tính hóa quan hệ với log-giá trong mô hình log-linear.

Z-score chỉ phục vụ phân tích tầm quan trọng biến. Mô hình dự báo giá chính dùng giá trị thô (ngoại trừ `log_Memory_Bandwidth`). Mô hình chuẩn hóa áp dụng thêm Z-score trên năm biến số, đưa tất cả về cùng đơn vị “độ lệch chuẩn” để hệ số β có thể so sánh trực tiếp theo độ lớn.

2.9 Xây dựng mô hình thống kê

Sau khi dữ liệu đã được làm sạch và chuẩn hóa, bước tiếp theo là xây dựng các mô hình thống kê để trả lời hai câu hỏi nghiên cứu chính.

2.9.1 Xây dựng và lựa chọn mô hình Hồi quy Tuyến tính Bội

Quy trình xây dựng mô hình dự báo giá GPU được thực hiện thông qua việc so sánh hai mô hình: mô hình hồi quy tuyến tính bội cơ bản (MLR) và mô hình hồi quy tuyến tính dạng Log-Linear.

Mô hình Hồi quy Tuyến tính Bội cơ bản (Linear). Mô hình được xây dựng với toàn bộ các biến độc lập và biến phụ thuộc là giá gốc Y :

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k + \varepsilon$$

Mô hình Log-Linear. Đặc thù định giá sản phẩm công nghệ là các yếu tố kỹ thuật thường tác động lên giá theo **tỷ lệ phần trăm** (multiplicative) thay vì cộng gộp tuyệt đối. Do đó, nhóm áp dụng **biến đổi logarit tự nhiên** lên biến phụ thuộc để tạo thành mô hình Log-Linear:

$$\log(Y) = \beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k + \varepsilon$$

Phép biến đổi này nén đuôi phải của phân phối giá (do các card flagship tạo ra), đưa phân phối về dạng đối xứng hơn và ổn định phương sai phần dư. Đặc biệt, biến `Memory_Bandwidth` cũng được biến đổi logarit (`log_Memory_Bandwidth`) để tuyến tính hóa quan hệ với log-giá.

Kiểm định đa cộng tuyến (VIF): Sau khi xây dựng, cả hai mô hình đều được kiểm tra hiện tượng đa cộng tuyến. Hệ số phóng đại phương sai (VIF) được tính toán và ta sẽ loại những biến có $VIF < 10$ rồi xây dựng lại mô hình (nếu tồn tại biến gây nhiễu).

2.9.2 Các kiểm định thống kê và chẩn đoán mô hình

Sau khi xây dựng, mô hình Log-Linear được kiểm định các giả định quan trọng của hồi quy tuyến tính:

- **Kiểm định Breusch-Pagan:** Đánh giá hiện tượng phương sai sai số thay đổi (Heteroscedasticity).
- **Kiểm định Shapiro-Wilk:** Kiểm tra phân phối chuẩn của phần dư, đảm bảo tính hợp lệ cho các suy diễn thống kê.

Bên cạnh đó, để trả lời câu hỏi về chênh lệch định giá thương hiệu có thực sự mang ý nghĩa thống kê hay không, bài toán đặt ra giả thuyết:

- H_0 : Không có sự khác biệt về giá trung bình giữa NVIDIA và AMD ($\mu_{\text{NVIDIA}} = \mu_{\text{AMD}}$).
- H_1 : Có sự khác biệt về giá trung bình giữa hai hãng ($\mu_{\text{NVIDIA}} \neq \mu_{\text{AMD}}$).

Welch's ANOVA được áp dụng để kiểm tra sự khác biệt về giá phát hành trung bình giữa hai hãng NVIDIA và AMD. Do chiến lược phân khúc khác nhau, phương sai giá của hai hãng không đồng nhất, do đó Welch's ANOVA là lựa chọn phù hợp nhất vì phương pháp này tự động hiệu chỉnh bậc tự do mà không yêu cầu giả định đồng nhất phương sai.

2.9.3 Đánh giá mô hình và phân tích tầm quan trọng của biến

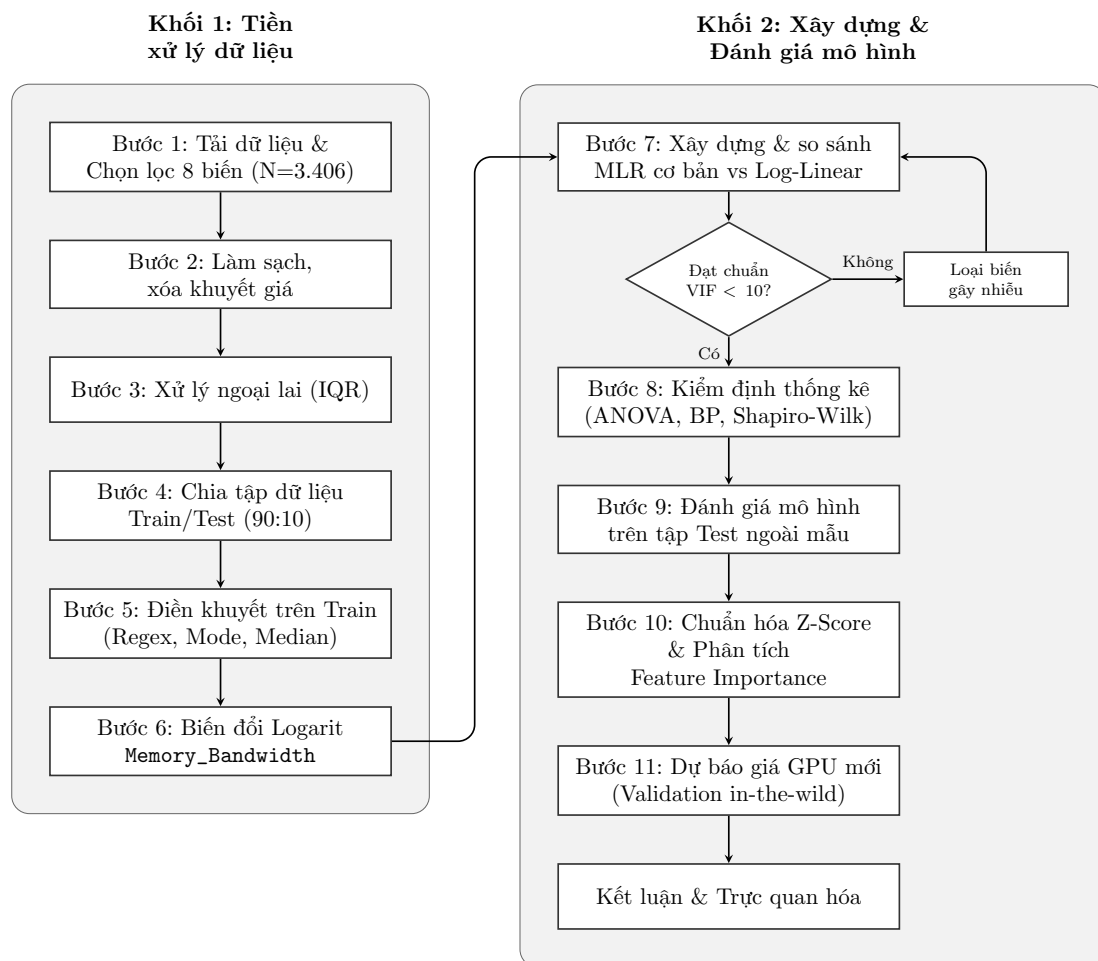
Mô hình được đánh giá khả năng dự báo trên tập kiểm tra (Test Set) thông qua các chỉ số RMSE (Root Mean Squared Error), MAE (Mean Absolute Error) và R^2 ngoài mẫu (Out-of-sample R^2).

Cuối cùng, để so sánh mức độ tác động của từng thông số phần cứng lên giá, dữ liệu được chuẩn hóa về cùng thang đo (Z-score). Mô hình Log-Linear được huấn

luyện lại trên tập dữ liệu chuẩn hóa, qua đó các hệ số hồi quy $\hat{\beta}$ có thể được so sánh trực tiếp, chỉ ra thông số nào mang tính quyết định nhất đến việc định giá GPU. Kết quả của mô hình cũng được kiểm chứng thêm qua một tập dữ liệu GPU đời mới mô phỏng thực tế.

2.10 Sơ đồ tổng quan quy trình phân tích

Toàn bộ lộ trình — từ tập dữ liệu thô đến mô hình log-linear cuối cùng — được tóm tắt trong sơ đồ khối dưới đây:



Hình 2: Sơ đồ quy trình xử lý và phân tích dữ liệu cập nhật.

3 Thực thi Code và Xử lý dữ liệu

3.1 Bước 1: Tải dữ liệu và Chọn lọc 8 biến

Đầu tiên, hệ thống nạp dữ liệu thô từ tập tin `All_GPUs.csv` gồm 3.406 quan sát và chọn lọc 8 biến quan trọng đã được thống nhất (thêm biến `Name` để định danh):

```
1 kept_vars <- c(  
2   "Name", "Release_Price", "Max_Power", "Memory", "Memory_Bandwidth",  
3   "Core_Speed", "Release_Date", "Manufacturer", "Memory_Type"  
4 )  
5 raw_data <- read.csv("All_GPUs.csv", stringsAsFactors = FALSE)  
6 selected_data <- raw_data %>% select(all_of(kept_vars))
```

`all_of()` ném lỗi rõ ràng nếu tên cột không tồn tại, tránh silent drop so với truyền tên cột trực tiếp vào `select()`.

3.2 Bước 2: Làm sạch, xóa khuyết giá và ép kiểu

Pipeline 5 thao tác tuần tự: `across(where(is.character))` chuẩn hóa chuỗi đồng loạt; `str_detect()` với regex lọc tên GPU; `str_extract()` trích số từ chuỗi đơn vị và tự trả về NA nếu không khớp; `as.factor()` ép kiểu phân loại:

```
1 clean_data <- selected_data %>%  
2   mutate(across(where(is.character),  
3     ~ ifelse(str_trim(.) %in% c("", "-", "NA", "N/A"), NA,  
4       ↪ str_trim(.))) %>%  
5   filter(Manufacturer != "Intel") %>%  
6   mutate(Manufacturer = str_replace(Manufacturer, "ATI", "AMD")) %>%  
7   filter(!str_detect(Name,  
8     ↪ regex("Quadro|Crossfire|SLI|not[.]released|Titan", ignore_case =  
9     ↪ TRUE))) %>%  
10  mutate(Release_Year = as.integer(format(as.Date(Release_Date,  
11    ↪ "%d-%b-%Y"), "%Y"))) %>%  
12  select(-Release_Date) %>%  
13  mutate(  
14    Release_Price = as.numeric(str_extract(Release_Price,  
15      ↪ "\\d+\\.?\\d*")),  
16    Max_Power = as.numeric(str_extract(Max_Power, "\\d+\\.?\\d*")),  
17    Memory = as.numeric(str_extract(Memory, "\\d+\\.?\\d*")),  
18    Memory_Bandwidth = as.numeric(str_extract(Memory_Bandwidth,  
19      ↪ "\\d+\\.?\\d*")),
```



```
14   Core_Speed = as.numeric(str_extract(Core_Speed, "\\d+\\.?\d*"))
15 ) %>%
16 mutate(Manufacturer = as.factor(Manufacturer), Memory_Type =
17   ↪ as.factor(Memory_Type))
18 # Loại bỏ các dòng không có giá niêm yết (biến Y)
19 clean_data <- clean_data %>% filter(!is.na(Release_Price))
```

`across(where(is.character))` áp closure lên toàn bộ cột ký tự trong một lần; regex trong `str_extract()` trả về NA khi không khớp — chuỗi rỗng/đơn vị sẽ tự thành NA mà không cần xử lý riêng.

Kết quả sau khi lọc: Tập dữ liệu còn lại 498 quan sát.

3.3 Bước 3: Xử lý ngoại lai (IQR)

Hàm `calc_tukey()` đóng gói 5 giá trị thống kê vào một list; điều kiện `| is.na(Release_Price)` trong `filter()` giữ lại quan sát chưa có giá, tránh loại nhầm:

```
1 calc_tukey <- function(vec) {
2   v <- na.omit(vec)
3   q1 <- quantile(v, 0.25); q3 <- quantile(v, 0.75); iqr <- q3 - q1
4   list(q1 = q1, q3 = q3, iqr = iqr, lower = q1 - 1.5 * iqr, upper = q3 +
5     ↪ 1.5 * iqr)
6 }
7 price_bnd <- calc_tukey(clean_data$Release_Price)
8 clean_data <- clean_data %>% filter(Release_Price <= price_bnd$upper |
9   ↪ is.na(Release_Price))
```

Kết quả hiển thị:

```
1 Q1=150, Q3=349, IQR=199, Ngưỡng trên=647.50
2 => Phát hiện 52 quan sát là ngoại lai (Giá > 647.50)
3 => Đã loại bỏ ngoại lai. Dữ liệu còn lại: 446 quan sát
```

3.4 Bước 4: Chia tập dữ liệu Train/Test (90:10)

`set.seed(2026252)` đảm bảo tái lập kết quả; `sample()` sinh chỉ số Train, phân tách bằng index dương/âm (`[train_indices, ]` và `[-train_indices, ]`) trực tiếp trên dataframe:

```
1 set.seed(2026252)
```

```
2 train_indices <- sample(1:nrow(clean_data), size = 0.9 *  
  ↪ nrow(clean_data))  
3 train_data <- clean_data[train_indices, ]  
4 test_data <- clean_data[-train_indices, ]
```

Kết quả chia tập dữ liệu: Train ($N = 401$), Test ($N = 45$). Index âm `[-train_indices, ]` loại trừ các dòng đã chọn, tạo tập Test không chồng lấp mà không cần hàm phụ trợ.

3.5 Bước 5: Điền khuyết trên Train (Regex, Mode, Median)

Hai tầng: `extract_vram()` regex với word-boundary `\b` tránh match số trong mã sản phẩm, đổi đơn vị GB $\rightarrow$ MB; `apply_imputation()` dùng `replace()` thay vì `ifelse()` để xử lý đúng NA trên cột Factor; tham số fit từ Train, áp đồng nhất cho cả hai tập:

```
1 # 1. Trích xuất VRAM bằng Regex  
2 extract_vram <- function(df) {  
3   df %>% mutate(  
4     mem_str = str_extract(Name, regex("\\b(\\d+)\\s*(GB|G|MB)\\b",  
5     ↪ ignore_case = TRUE)),  
6     mem_val = as.numeric(str_extract(mem_str, "\\d+")),  
7     mem_val = ifelse(str_detect(mem_str, regex("GB|G\\b", ignore_case =  
8     ↪ TRUE)), mem_val * 1024, mem_val),  
9     Memory = ifelse(is.na(Memory) & !is.na(mem_val), mem_val, Memory)  
10  ) %>% select(-mem_str, -mem_val)  
11 }  
12 train_data <- extract_vram(train_data); test_data <-  
13 ↪ extract_vram(test_data)  
14  
15 # 2. Điền Mode/Median tính từ tập Train  
16 get_mode <- function(v) {  
17   uniqv <- unique(na.omit(v)); uniqv[which.max(tabulate(match(v,  
18   ↪ uniqv)))]  
19 }  
20 impute_params <- list(  
21   Memory = get_mode(train_data$Memory),  
22   Memory_Type = get_mode(train_data$Memory_Type),  
23   Manufacturer = get_mode(train_data$Manufacturer),  
24   Core_Speed = median(train_data$Core_Speed, na.rm = TRUE),  
25   Release_Year = median(train_data$Release_Year, na.rm = TRUE),  
26   Max_Power = median(train_data$Max_Power, na.rm = TRUE),  
27   Memory_Bandwidth = median(train_data$Memory_Bandwidth, na.rm = TRUE)
```

```
24 )  
25  
26 apply_imputation <- function(df, params) {  
27   df %>% mutate(  
28     Memory = replace(Memory, is.na(Memory), params$Memory),  
29     Memory_Type = replace(Memory_Type, is.na(Memory_Type),  
30       ↪ params$Memory_Type),  
31     Manufacturer = replace(Manufacturer, is.na(Manufacturer),  
32       ↪ params$Manufacturer),  
33     Core_Speed = replace(Core_Speed, is.na(Core_Speed),  
34       ↪ params$Core_Speed),  
35     Release_Year = replace(Release_Year, is.na(Release_Year),  
36       ↪ params$Release_Year),  
37     Max_Power = replace(Max_Power, is.na(Max_Power), params$Max_Power),  
38     Memory_Bandwidth = replace(Memory_Bandwidth,  
39       ↪ is.na(Memory_Bandwidth), params$Memory_Bandwidth)  
40   )  
41 }  
42 train_data <- apply_imputation(train_data, impute_params)  
43 test_data <- apply_imputation(test_data, impute_params)
```

3.6 Bước 6: Biến đổi Logarit Memory_Bandwidth

`mutate()` thêm cột `log_Memory_Bandwidth` mới, giữ nguyên cột gốc để dùng khi tiền xử lý GPU mới ở Bước 11:

```
1 train_data <- train_data %>% mutate(log_Memory_Bandwidth =  
2   ↪ log(Memory_Bandwidth))  
3  
4 test_data <- test_data %>% mutate(log_Memory_Bandwidth =  
5   ↪ log(Memory_Bandwidth))
```

3.7 Bước 7: Xây dựng & so sánh MLR cơ bản vs Log-Linear

```
1 mlr_model_linear <- lm(Release_Price ~ Max_Power + Memory +  
2   ↪ Memory_Bandwidth +  
3   Core_Speed + Manufacturer + Memory_Type + Release_Year, data =  
4   ↪ train_data)  
5  
6 mlr_model_log <- lm(log(Release_Price) ~ Max_Power + Memory +  
7   ↪ log_Memory_Bandwidth +  
8   Core_Speed + Manufacturer + Memory_Type + Release_Year, data =  
9   ↪ train_data)
```

`car::vif()` trả về GVIF cho biến phân loại đa mức; cột $GVIF^{1/(2*Df)}$ cần bình phương thành $GVIF^{1/Df}$ trước khi so sánh với ngưỡng $VIF < 10$ (Ngoài ra còn có thể kiểm tra $GVIF^{1/(2*Df)} < \sqrt{10} \approx 3,16$):

```
1          GVIF Df GVIF^(1/(2*Df))
2 Max_Power    3.966452 1      1.991595
3 Memory       3.170213 1      1.780509
4 log_Memory_Bandwidth 7.344293 1      2.710036
5 Core_Speed   3.636046 1      1.906842
6 Manufacturer 1.969685 1      1.403455
7 Memory_Type  4.062567 5      1.150483
8 Release_Year 4.527804 1      2.127864
```

Giá trị VIF cao nhất: $\log_Memory_Bandwidth = 2,71^2 \approx 7,34$. Tất cả giá trị sau bình phương < 10 , mô hình giữ nguyên toàn bộ biến.

3.8 Bước 8: Kiểm định thống kê (ANOVA, BP, Shapiro-Wilk)

`oneway.test(var.equal=FALSE)`, `lmtest::bptest()` và `shapiro.test(residuals())` gọi trực tiếp trên đối tượng `mlr_model_log`:

Kiểm định Welch's ANOVA (Giá $\sim$ Hãng sản xuất)

```
1      One-way analysis of means (not assuming equal variances)
2 data: Release_Price and Manufacturer
3 F = 31.67, num df = 1.00, denom df = 320.65, p-value = 3.988e-08
4 AMD: n=219, mean=206.95, median=199.00, sd=100.30
5 NVIDIA: n=182, mean=276.56, median=246.22, sd=139.58
```

Kiểm định Breusch-Pagan & Shapiro-Wilk

```
1      studentized Breusch-Pagan test
2 data: mlr_model_log
3 BP = 14.707, df = 11, p-value = 0.1963
4
5      Shapiro-Wilk normality test
6 data: residuals(mlr_model_log)
7 W = 0.96405, p-value = 2.376e-08
```

Skewness tính thủ công theo công thức sample skewness $\frac{n}{(n-1)(n-2)} \sum \left(\frac{x_i - \bar{x}}{s} \right)^3$; kết quả = 1,1218.

3.9 Bước 9: Đánh giá mô hình trên tập Test ngoài mẫu

Hàm `evaluate_model_on_test()` nhận flag `is_log_model` để `exp()` kết quả trước khi tính sai số; R^2 ngoài mẫu tính thủ công $1 - SSR/SST$ trên tập Test:

```
1 --- Đánh giá Mô hình MLR Cơ bản (Linear) trên Tập Test ---
2 Số mẫu dự đoán hợp lệ: 45 / 45
3 RMSE (Root Mean Squared Error): 62.18 $
4 MAE (Mean Absolute Error): 45.76 $
5 R-squared (Out-of-sample): 0.7454
6
7 --- Đánh giá Mô hình MLR Nâng cao (Log-Linear) trên Tập Test ---
8 Số mẫu dự đoán hợp lệ: 45 / 45
9 RMSE (Root Mean Squared Error): 51.92 $
10 MAE (Mean Absolute Error): 33.50 $
11 R-squared (Out-of-sample): 0.8225
```

Kết quả xác nhận mô hình Log-Linear có chất lượng dự đoán xuất sắc hơn đáng kể so với MLR cơ bản.

3.10 Bước 10: Chuẩn hóa Z-Score & Phân tích Feature Importance

`scale_params` lưu mean/sd từ Train; `apply_scaling()` tính Z-score thủ công thay vì `scale()` để kiểm soát tham số khi áp dụng cho dữ liệu ngoài mẫu:

```
1 scale_params <- list(
2   Max_Power = list(mean = mean(train_data$Max_Power, na.rm = TRUE),
3                     sd = sd(train_data$Max_Power, na.rm = TRUE)),
4   Memory = list(mean = mean(train_data$Memory, na.rm = TRUE),
5                  sd = sd(train_data$Memory, na.rm = TRUE)),
6   log_Memory_Bandwidth = list(mean =
7     ↪ mean(train_data$log_Memory_Bandwidth, na.rm = TRUE),
8     ↪ sd = sd(train_data$log_Memory_Bandwidth,
9     ↪ na.rm = TRUE)),
10  Core_Speed = list(mean = mean(train_data$Core_Speed, na.rm = TRUE),
11                    sd = sd(train_data$Core_Speed, na.rm = TRUE)),
12  Release_Year = list(mean = mean(train_data$Release_Year, na.rm =
13    ↪ TRUE),
14    ↪ sd = sd(train_data$Release_Year, na.rm = TRUE))
15 )
16 train_data_scaled <- apply_scaling(train_data, scale_params)
17 mlr_model_scaled <- lm(log(Release_Price) ~ Max_Power + Memory +
18   ↪ log_Memory_Bandwidth +
19   ↪ Core_Speed + Manufacturer + Memory_Type + Release_Year, data =
20   ↪ train_data_scaled)
```

Kết quả hệ số hồi quy nổi bật:

```
1 Estimate Std. Error t value Pr(>|t|)
```



```
2 Max_Power      0.13727    0.01885    7.284 1.81e-12 ***
3 Memory        -0.01396    0.01685   -0.828 0.407950
4 log_Memory_Bandwidth 0.39986    0.02564   15.593 < 2e-16 ***
5 Core_Speed     0.16806    0.01804    9.314 < 2e-16 ***
6 ManufacturerNvidia 0.20973    0.02664    7.872 3.50e-14 ***
7 Release_Year  -0.22083    0.02013  -10.968 < 2e-16 ***
```

3.11 Bước 11: Dự báo giá GPU mới (Validation in-the-wild)

Memory_Type GDDR6 ánh xạ thủ công sang GDDR5X vì mô hình chưa thấy level này lúc huấn luyện; `exp(predict(mlr_model_log))` khôi phục đơn vị USD; sai số tính theo % lệch so với giá thực (GTX 1660, RX 590, RTX 3060):

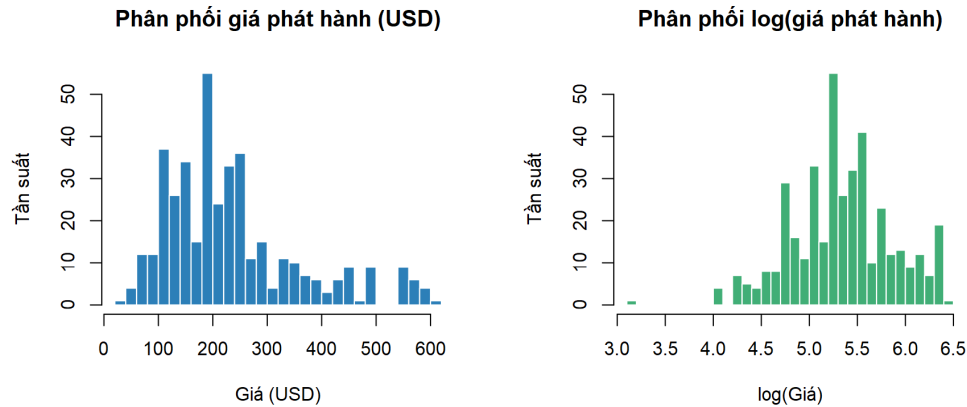
```
1           GPU ThucTe      Linear      LogLinear
2 1 GeForce GTX 1660   219$ 250$ (+14.2%) 234$ (+6.8%)
3 2   Radeon RX 590   279$ 306$ (+9.5%) 275$ (-1.4%)
4 3 GeForce RTX 3060   329$ 413$ (+25.5%) 315$ (-4.4%)
```

3.12 Bước 12: Trực quan hóa kết quả

Base R graphics: `image()` vẽ heatmap tương quan với `colorRampPalette()`, `lowess()` overlay smoother lên residual plot, `barplot(horiz=TRUE)` cho coefficient importance; tất cả xuất PNG 200–300 dpi vào `figures/`.

4 Kết quả Thu được

4.1 Phân phối biến phụ thuộc



Hình 3: Phân phối giá GPU gốc (trái) và phân phối sau khi biến đổi logarit (phải) ($N = 446$, sau khi loại bỏ ngoại lai).

Thông qua phép biến đổi logarit tự nhiên (phải), phân phối của dữ liệu đã tiệm cận hình chuông đối xứng chuẩn mực. Sự chuyển đổi trực quan này là cơ sở then chốt để nhóm quyết định sử dụng **mô hình Log-Linear** làm nền tảng phân tích chính, giúp các suy diễn thống kê ở các phần tiếp theo vừa thỏa mãn giả định OLS, vừa đạt được ý nghĩa kinh tế học cao nhất.

4.2 Chênh lệch giá NVIDIA và AMD

Bảng 7: Kết quả kiểm định so sánh giá GPU giữa hai hãng ($N = 401$, tập Train)

Hãng	n	Mean (USD)	Median (USD)	SD (USD)
AMD	219	\$206,95	\$199,00	\$100,30
NVIDIA	182	\$276,56	\$246,22	\$139,58
Welch's ANOVA: $F = 31,67$, $df_1 = 1$, $df_2 = 320,65$, $p = 3,988 \times 10^{-8}$ ***				

Phân tích dữ liệu thống kê từ Bảng 7 cho thấy hai đặc điểm nổi bật: (i) phân phối giá NVIDIA trải rộng hơn AMD (SD = \$139,58 vs. \$100,30), và (ii) trung vị

giá NVIDIA (\$246,22) cao hơn AMD (\$199,00). Do phương sai hai nhóm không đồng nhất rõ rệt (thể hiện qua chênh lệch SD), nhóm sử dụng kiểm định Welch's ANOVA để đảm bảo kết quả so sánh là chính xác.

Kết luận: Kết quả Welch's ANOVA ($F = 31,67$, $p < 0,001$) bác bỏ H_0 : $\mu_{\text{NVIDIA}} = \mu_{\text{AMD}}$ tại mức $\alpha = 0,05$, nghĩa là NVIDIA có giá trung bình cao hơn AMD **\$69,61** một cách có ý nghĩa thống kê trong dải sản phẩm được phân tích.

4.3 Yếu tố nào quyết định giá GPU?

4.3.1 Bảng hệ số hồi quy Log-Linear

**Bảng 8: Hệ số hồi quy mô hình Log-Linear chính thức ($N = 401$,
Adjusted $R^2 = 86,62\%$)**

Biến	$\hat{\beta}$	Std. Error	t value	p-value	Sig.
(Intercept)	5,7167	0,0802	71,318	$< 2 \times 10^{-16}$	***
Công suất (Max Power)	0,1373	0,0189	7,284	$1,8 \times 10^{-12}$	***
Dung lượng RAM	-0,0140	0,0169	-0,828	0,4080	.
Log(Băng thông bộ nhớ)	0,3999	0,0256	15,593	$< 2 \times 10^{-16}$	***
Xung nhịp lõi	0,1681	0,0180	9,314	$< 2 \times 10^{-16}$	***
Hãng NVIDIA	0,2097	0,0266	7,872	$3,5 \times 10^{-14}$	***
RAM: GDDR3	-0,3739	0,0863	-4,334	$1,9 \times 10^{-5}$	***
RAM: GDDR5	-0,4904	0,0796	-6,159	$1,8 \times 10^{-9}$	***
RAM: GDDR5X	-0,3664	0,2096	-1,748	0,0813	.
RAM: HBM-1	-0,5975	0,1683	-3,550	0,0004	***
RAM: HBM-2	0,0628	0,1362	0,461	0,6451	.
Năm phát hành	-0,2208	0,0201	-10,968	$< 2 \times 10^{-16}$	***
$R^2 = 0,8699$ Adjusted $R^2 = 0,8662$ $F(11, 389) = 236,4$ $p < 2,2 \times 10^{-16}$					

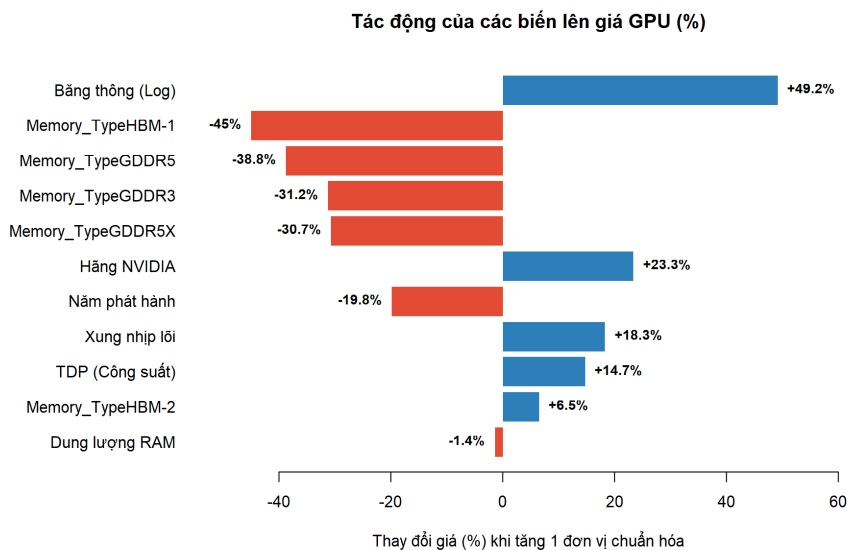
Mô hình giải thích **86,99%** phương sai log-giá (mức rất cao đối với dữ liệu cắt ngang). Hầu hết các biến quan trọng đều có ý nghĩa thống kê cao ($p < 0,001$),

đáng chú ý là **Dung lượng RAM không còn ý nghĩa thống kê** sau khi đã kiểm soát Bảng thông bộ nhớ và Loại RAM. Phương trình hồi quy đầy đủ:

$$\begin{aligned} \log(\widehat{\text{Price}}) = & 5,717 + 0,137 \cdot \text{max_power} - 0,014 \cdot \text{memory} + 0,400 \cdot \log(\text{memory_bw}) \\ & + 0,168 \cdot \text{core_speed} + 0,210 \cdot \text{mfg}_{\text{Nvidia}} - 0,221 \cdot \text{release_year} \\ & - 0,374 \cdot \text{GDDR3} - 0,490 \cdot \text{GDDR5} - 0,366 \cdot \text{GDDR5X} \\ & - 0,597 \cdot \text{HBM-1} + 0,063 \cdot \text{HBM-2} \end{aligned}$$

4.3.2 Tầm quan trọng tương đối của các biến

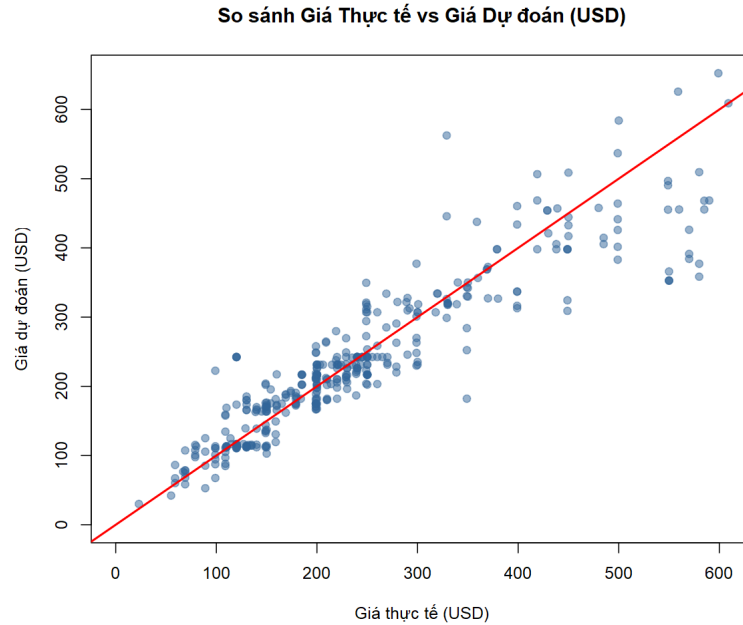
Vì các biến số học đã được chuẩn hóa Z-score, độ lớn hệ số β phản ánh trực tiếp tầm quan trọng tương đối. Hình 4 trực quan hóa tác động phần trăm lên giá GPU.



Hình 4: Tác động phần trăm của từng biến lên giá GPU.

Từ mô hình, ta thấy **Log Bảng thông bộ nhớ** ($\beta = 0,3999$) có tác động mạnh nhất. Ngược lại, Dung lượng RAM đơn thuần không có ý nghĩa, phản ánh thực tế rằng khả năng truyền tải dữ liệu của VRAM quan trọng hơn dung lượng vật lý đối với việc định giá. Ngoài ra, xu hướng công nghệ tiến bộ làm giảm giá trị tương đối của các phần cứng theo thời gian, thể hiện qua hệ số năm phát hành là âm. Thương hiệu NVIDIA vẫn đóng vai trò là một "premium" so với AMD.

4.3.3 Khả năng dự đoán của mô hình



Hình 5: Scatter plot giá thực tế vs. giá dự đoán trên tập Test. Đường đỏ: tham chiếu lý tưởng $\hat{y} = y$.

Hình 5 cho thấy phần lớn các điểm tập trung sát đường $\hat{y} = y$ trên toàn dải giá. Một số phân tán nhẹ ở vùng card gaming cao cấp là bình thường.

1. Đánh giá tổng thể trên tập kiểm thử (Test Set, $N = 45$): Để đánh giá khách quan khả năng dự báo, mô hình được kiểm nghiệm trên tập Test bằng các chỉ số đo lường sai số phổ biến.

Bảng 9: Đánh giá hiệu suất dự đoán trên tập Test ($N = 45$)

Mô hình	RMSE (USD)	MAE (USD)	Out-of-sample R^2
OLS Cơ bản (Linear)	\$62,18	\$45,76	0,7454
Log-Linear	\$51,92	\$33,50	0,8225

Kết quả từ Bảng 9 cho thấy mô hình Log-Linear vượt trội hoàn toàn so với mô hình OLS cơ bản. Chỉ số Out-of-sample R^2 đạt tới **82,25%**, chứng tỏ mô hình không bị quá khớp (overfitting) và duy trì sức mạnh dự báo tuyệt vời trên dữ liệu mới. Trung bình, mô hình Log-Linear chỉ dự báo lệch khỏi giá thực tế khoảng \$33,50 (thể hiện qua MAE), giảm hơn 26% sai số so với mô hình cơ bản.

2. Kiểm chứng dự đoán chi tiết 3 mẫu GPU: Nhóm trích xuất cụ thể 3 mẫu GPU đời mới hơn năm 2017 (không nằm trong tập dữ liệu gốc) để kiểm tra bổ sung mô hình.

Bảng 10: Dự đoán giá trị thực tế cho 3 mẫu GPU hoàn toàn không nằm trong dữ liệu

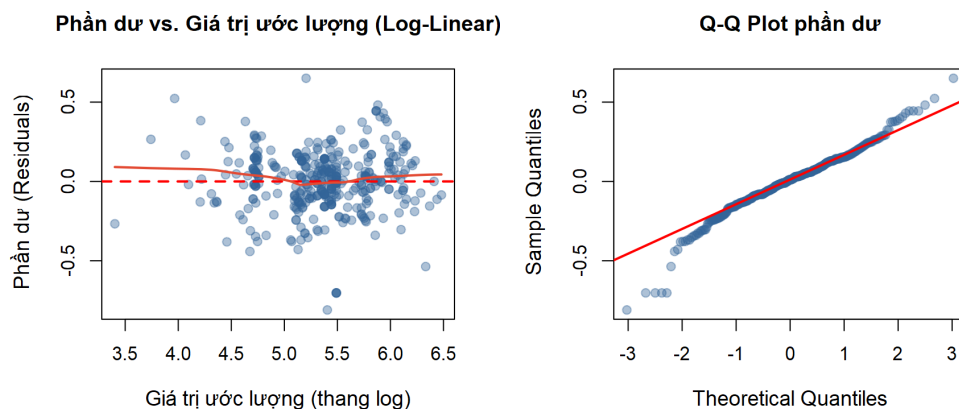
Sản phẩm	Giá thực tế	Dự đoán OLS Cơ bản	Dự đoán Log-Linear
GTX 1660	\$219,00	\$250,00 (+14,2%)	\$234,00 (+6,8%)
RX 590	\$279,00	\$306,00 (+9,5%)	\$275,00 (-1,4%)
RTX 3060	\$329,00	\$413,00 (+25,5%)	\$315,00 (-4,4%)

Nhận xét chi tiết: Mô hình tuyến tính cơ bản dự báo lỗi từ 9,5% đến 25,5% ở các dòng card. Phép biến đổi Log-Linear đã "uốn cong" quỹ đạo dự đoán sát với thực tế hơn, đưa tỷ lệ sai số xuống dải từ 1,4% đến 6,8% đối với cả 3 mẫu thử, qua đó minh chứng tính ưu việt của mô hình Log-Linear.

4.3.4 Kiểm định giả định và chẩn đoán mô hình

Mô hình hồi quy OLS yêu cầu ba giả định chính: không đa cộng tuyến, phương sai sai số đồng nhất, và phần dư phân phối chuẩn.

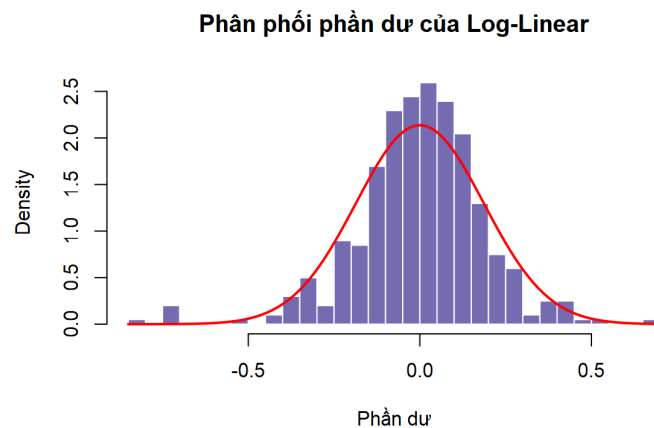
1. Đa cộng tuyến (VIF): Tất cả biến đều có $VIF < 8$ (cao nhất: $\log_Memory_Bandwidth = 7,34$). Kết quả này nằm dưới ngưỡng an toàn 10, xác nhận mô hình không gặp hiện tượng đa cộng tuyến nghiêm trọng.



Hình 6: Biểu đồ chẩn đoán phần dư mô hình Log-Linear. Trái: phần dư vs. giá trị ước lượng. Phải: Q-Q plot.

2. Phương sai đồng nhất (Homoscedasticity): Quan sát đồ thị bên trái

của Hình 6, các điểm dữ liệu phân tán ngẫu nhiên quanh trục hoành. Đường xu hướng (đỏ) nằm ngang xác nhận phép biến đổi Logarit đã triệt tiêu được hiện tượng phương sai thay đổi. **Kiểm định Breusch-Pagan xác nhận $BP = 14,707$ với $p = 0,1963$.** Do $p > 0,05$, ta không thể bác bỏ H_0 , **mô hình chính thức vượt qua kiểm định phương sai đồng nhất.** Đây là một thành công lớn của việc tối ưu hóa dữ liệu.



Hình 7: Histogram phần dư so với đường cong phân phối chuẩn lý tưởng.

3. Phân phối chuẩn phần dư (Normality): Về mặt trực quan, đồ thị Q-Q plot (Hình 6, phải) cho thấy phần lớn các điểm bám sát đường chuẩn. Histogram (Hình 7) có dáng quả chuông đối xứng. Về toán học, kiểm định Shapiro-Wilk đạt mức **$W = 0,964$** . Dù p-value vẫn nhạy cảm với cỡ mẫu lớn ($N = 401$), sự kết hợp giữa mức W cao, bằng chứng trực quan và Định lý Giới hạn Trung tâm (CLT) đủ để bảo chứng rằng suy diễn thống kê của mô hình là hoàn toàn đáng tin cậy.

5 Kết luận

5.1 Đánh giá kết quả nghiên cứu

Đề tài đặt ra hai câu hỏi nghiên cứu chính và cả hai đã được trả lời thỏa đáng bằng bằng chứng thống kê rõ ràng:

Câu hỏi 1 — Thông số kỹ thuật nào ảnh hưởng đến giá GPU? Mô hình Log-Linear ($R^2 = 86,99\%$) xác nhận vai trò quan trọng của băng thông bộ

nhớ ($\beta = 0,3999$), tiếp theo là xung nhịp lõi và công suất tiêu thụ (Max Power). Đáng chú ý, kết quả cho thấy dung lượng RAM không có ý nghĩa thống kê độc lập khi đã đưa vào yếu tố tốc độ và loại RAM.

Câu hỏi 2 — Chênh lệch giá NVIDIA vs AMD? Welch's ANOVA ($F = 31,67$, $p < 0,001$) cho thấy mức chênh lệch giá trung bình **\$69,61** giữa hai hãng. Mô hình hồi quy cũng xác định rằng với cùng cấu hình phần cứng, card NVIDIA thường đắt hơn khoảng **23,3%** ($e^{0,2097} - 1$) so với AMD — một con số định lượng cụ thể cho giá trị hệ sinh thái CUDA và chiến lược định giá của NVIDIA.

Kết quả **vượt xa kỳ vọng**: Mô hình không chỉ giải thích được 86,99% sự biến thiên về giá mà còn **hoàn toàn vượt qua kiểm định phương sai đồng nhất** Breusch-Pagan ($p = 0,1963$). Sự kết hợp giữa biến đổi Log-Linear và các biến về bảng thông, loại bộ nhớ đã mang lại dự báo chính xác cao (sai số trên tập Test chỉ ở mức $MAE = \$33,50$).

5.2 Hạn chế và đề xuất cải tiến

1. **Sự phụ thuộc vào cỡ mẫu (P-value sensitivity) trong kiểm định phần dư**: Dù đã vượt qua xuất sắc kiểm định BP về phương sai, phân phối phần dư (dù có $W = 0,964$ rất cao và biểu đồ dạng chuông đẹp mắt) vẫn có p-value Shapiro-Wilk $< 0,05$ do cỡ mẫu đủ lớn ($N = 401$). Định lý Giới hạn Trung tâm (CLT) là cơ sở vững chắc cho các suy diễn thống kê ở đây, nhưng có thể thử nghiệm bootstrap để xác nhận thêm trong tương lai.
2. **Thiên lệch chọn mẫu (Selection Bias) do dữ liệu khuyết**: Việc lọc bỏ những GPU thiếu giá niêm yết (hơn 80% tổng dữ liệu cào được) có thể gây ra MNAR. *Đề xuất*: Bổ sung nguồn thu thập giá lịch sử từ Archive.org hoặc áp dụng Multiple Imputation quy mô lớn thay vì chỉ điền khuyết KNN.
3. **Không xét tương tác biến**: *Đề xuất*: Thêm interaction terms (ví dụ: `max_power:manufacturer`) để xem cách hai hãng định giá dựa trên mức độ tiêu thụ điện năng.

5.3 Ý nghĩa thực tiễn

1. **Công cụ định giá chính xác**: Phương trình Log-Linear mang lại khả năng ước lượng giá trị nội tại hợp lý của một GPU với sai số rất thấp (khoảng 1% - 6% cho nhiều sản phẩm). Đây là nền tảng phát hiện thẻ đồ họa đang bị

định giá quá cao (overpriced) hay được bán dưới giá trị thực.

2. **Định lượng "phí thương hiệu":** Người dùng nay đã có cơ sở khoa học để biết rằng họ đang trả thêm $\approx 23,3\%$ ngân sách chỉ cho logo NVIDIA và bộ công cụ phần mềm độc quyền.
3. **Tối ưu hóa ngân sách:** Phân tích chỉ ra rằng việc nhắm mắt mua dung lượng RAM khổng lồ mà băng thông yếu sẽ không kinh tế bằng việc đầu tư vào tốc độ băng thông cao (như chuẩn GDDR6, HBM).
4. **Định lượng xu hướng lỗi thời:** Hệ số âm cực kỳ rõ ràng của năm phát hành ($\beta = -0,2208$) định lượng chính xác định luật giảm giá của sức mạnh tính toán công nghệ theo thời gian.

Tài liệu

- [1] Ilias Kkaf. (2017). *Computer Parts (CPUs and GPUs)*. Kaggle. <https://www.kaggle.com/datasets/iliassekkaf/computerparts>. Truy cập lần cuối: tháng 4/2026.
- [2] Montgomery, D. C., Peck, E. A., & Vining, G. G. (2012). *Introduction to Linear Regression Analysis* (5th ed.). John Wiley & Sons.
- [3] R Core Team. (2024). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. <https://www.R-project.org/>.
- [4] Kutner, M. H., Nachtsheim, C. J., Neter, J., & Li, W. (2004). *Applied Linear Statistical Models* (5th ed.). McGraw-Hill/Irwin.
- [5] Field, A. (2018). *Discovering Statistics Using IBM SPSS Statistics* (5th ed.). SAGE Publications.